

Punto 5.2 — Configuración de tokens de concurrencia

Contexto del proyecto

Este punto continúa directamente M05/PROYECTO/5.1. La carpeta M05/PROYECTO/5.2 contiene la solución completa AceriaData.sln y conserva las capas y migraciones existentes. Este punto introduce la migración real M5_5_2_ConcurrencyTokens.

Objetivo práctico

Configurar rowversion y un token de propiedad, generar la migración real y demostrar el conflicto sobre SQL Server LocalDB.

Paso 1: Abrir el estado completo del punto

```
cd M05/PROYECTO/5.2
dotnet restore AceriaData.sln
```

La restauración se ejecuta sobre la solución local del punto. No es necesario copiar archivos desde otros puntos: la carpeta ya contiene el estado acumulativo completo.

Paso 2: Identificar los archivos principales

En este punto se trabajan principalmente estos archivos:

- src/AceriaData.Infrastructure/Persistence/Configurations/ConcurrencyTokensConfiguration.cs
- src/AceriaData.Infrastructure/Repositories/TokensConcurrenciaM5Repositorio.cs

Los archivos se estudian dentro de su capa real. Application conserva sus abstracciones y casos de uso; Infrastructure contiene EF Core y acceso a proveedor; Console compone y ejecuta; los tests se ubican fuera de las capas de producción cuando aparecen en 5.9 y 5.11.

Paso 3: Implementar y estudiar ConcurrencyTokensConfiguration.cs

```
public sealed class OrdenFabricacionConcurrencyConfiguration :
IEntityTypeConfiguration<OrdenFabricacion>
{
    public void Configure(EntityTypeBuilder<OrdenFabricacion> b) =>
        b.Property(x => x.RowVersion).IsRowVersion();
}

public sealed class PlanchaAceroConcurrencyConfiguration : IEntityTypeConfiguration<PlanchaAcero>
{
    public void Configure(EntityTypeBuilder<PlanchaAcero> b) =>
        b.Property(x => x.RowVersion).IsRowVersion();
}

public sealed class AleacionConcurrencyConfiguration : IEntityTypeConfiguration<Aleacion>
{
    public void Configure(EntityTypeBuilder<Aleacion> b) =>
        b.Property(x => x.RowVersion).IsRowVersion();
}

public sealed class DetalleOrdenConcurrencyConfiguration : IEntityTypeConfiguration<DetalleOrden>
{
    public void Configure(EntityTypeBuilder<DetalleOrden> b)
    {
        b.Property(x => x.EstadoDetalle)
            .IsRequired()
            .HasMaxLength(50)
            .HasDefaultValue("Pendiente")
            .IsConcurrencyToken();
    }
}
```

Explicación línea a línea — ConcurrencyTokensConfiguration.cs

Línea 1: `public sealed class OrdenFabricacionConcurrencyConfiguration :`
`IEntityTypeConfiguration<OrdenFabricacion>` → Declara la clase que encapsula la responsabilidad mostrada en este ejemplo.

Línea 2: `{` → Delimita el bloque sintáctico correspondiente.

Línea 3: `public void Configure(EntityTypeBuilder<OrdenFabricacion> b) =>` → Declara la operación pública que ejecutará o verificará este comportamiento.

Línea 4: `b.Property(x => x.RowVersion).IsRowVersion();` → Configura la propiedad como token rowversion generado por SQL Server y usado por EF Core para concurrencia optimista.

Línea 5: `}` → Delimita el bloque sintáctico correspondiente.

Línea 7: `public sealed class PlanchaAceroConcurrencyConfiguration :`
`IEntityTypeConfiguration<PlanchaAcero>` → Declara la clase que encapsula la responsabilidad mostrada en este ejemplo.

Línea 8: `{` → Delimita el bloque sintáctico correspondiente.

Línea 9: `public void Configure(EntityTypeBuilder<PlanchaAcero> b) =>` → Declara la operación pública que ejecutará o verificará este comportamiento.

Línea 10: `b.Property(x => x.RowVersion).IsRowVersion();` → Configura la propiedad como token rowversion generado por SQL Server y usado por EF Core para concurrencia optimista.

Línea 11: `}` → Delimita el bloque sintáctico correspondiente.

Línea 13: `public sealed class AleacionConcurrencyConfiguration :`
`IEntityTypeConfiguration<Aleacion>` → Declara la clase que encapsula la responsabilidad mostrada en este ejemplo.

Línea 14: `{` → Delimita el bloque sintáctico correspondiente.

Línea 15: `public void Configure(EntityTypeBuilder<Aleacion> b) =>` → Declara la operación pública que ejecutará o verificará este comportamiento.

Línea 16: `b.Property(x => x.RowVersion).IsRowVersion();` → Configura la propiedad como token rowversion generado por SQL Server y usado por EF Core para concurrencia optimista.

Línea 17: `}` → Delimita el bloque sintáctico correspondiente.

Línea 19: `public sealed class DetalleOrdenConcurrencyConfiguration : IEntityTypeConfiguration<DetalleOrden>` → Declara la clase que encapsula la responsabilidad mostrada en este ejemplo.

Línea 20: `{` → Delimita el bloque sintáctico correspondiente.

Línea 21: `public void Configure(EntityTypeBuilder<DetalleOrden> b)` → Declara la operación pública que ejecutará o verificará este comportamiento.

Línea 22: `{` → Delimita el bloque sintáctico correspondiente.

Línea 23: `b.Property(x => x.EstadoDetalle)` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 24: `.IsRequired()` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 25: `.HasMaxLength(50)` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 26: `.HasDefaultValue("Pendiente")` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 27: `.IsConcurrencyToken();` → Marca la propiedad existente como token de concurrencia para que su valor original participe en escrituras.

Línea 28: `}` → Delimita el bloque sintáctico correspondiente.

Línea 29: `}` → Delimita el bloque sintáctico correspondiente.

Paso 4: Implementar y estudiar TokensConcurrenciaM5Repositorio.cs

```
public RowVersionM5Dto DemostrarRowVersion()
{
    using var scopeA = _scopeFactory.CreateScope();
    using var scopeB = _scopeFactory.CreateScope();
    var contextA = scopeA.ServiceProvider.GetRequiredService<AceriaDbContext>();
    var contextB = scopeB.ServiceProvider.GetRequiredService<AceriaDbContext>();

    var ordenA = contextA.OrdenesFabricacion.Single(o => o.NumeroOrden == NumeroOrden);
    var ordenB = contextB.OrdenesFabricacion.Single(o => o.NumeroOrden == NumeroOrden);
    var inicialA = Convert.ToHexString(ordenA.RowVersion);
    var inicialB = Convert.ToHexString(ordenB.RowVersion);

    SqlCommandCounterInterceptor.Instance.Reset();
    ordenA.Cliente = "Cliente 5.2 A";
    contextA.SaveChanges();
    var despuesA = Convert.ToHexString(ordenA.RowVersion);

    ordenB.Estado = "EnProceso";
    var conflicto = false;
    try
    {
        contextB.SaveChanges();
    }
    catch (DbUpdateConcurrencyException)
    {
        conflicto = true;
    }

    return new RowVersionM5Dto(
        inicialA, inicialB, despuesA, conflicto,
        SqlCommandCounterInterceptor.Instance.SnapshotCommands());
}
```

Explicación línea a línea — TokensConcurrenciaM5Repositorio.cs

Línea 1: `public RowVersionM5Dto DemostrarRowVersion()` → Declara la operación pública que ejecutará o verificará este comportamiento.

Línea 2: `{` → Delimita el bloque sintáctico correspondiente.

Línea 3: `using var scopeA = _scopeFactory.CreateScope();` → Crea un ámbito de DI independiente; los servicios Scoped, incluido `AceriaDbContext`, no se comparten con otros actores.

Línea 4: `using var scopeB = _scopeFactory.CreateScope();` → Crea un ámbito de DI independiente; los servicios Scoped, incluido `AceriaDbContext`, no se comparten con otros actores.

Línea 5: `var contextA = scopeA.ServiceProvider.GetRequiredService<AceriaDbContext>();` → Resuelve el `DbContext` del ámbito y garantiza que la configuración de Infrastructure sea la misma que en la aplicación.

Línea 6: `var contextB = scopeB.ServiceProvider.GetRequiredService<AceriaDbContext>();` → Resuelve el `DbContext` del ámbito y garantiza que la configuración de Infrastructure sea la misma que en la aplicación.

Línea 8: `var ordenA = contextA.OrdenesFabricacion.Single(o => o.NumeroOrden == NumeroOrden);` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 9: `var ordenB = contextB.OrdenesFabricacion.Single(o => o.NumeroOrden == NumeroOrden);` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 10: `var inicialA = Convert.ToHexString(ordenA.RowVersion);` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 11: `var inicialB = Convert.ToHexString(ordenB.RowVersion);` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 13: `SqlCommandCounterInterceptor.Instance.Reset();` → Reinicia el contador para que solo se midan los comandos de este escenario.

Línea 14: `ordenA.Cliente = "Cliente 5.2 A";` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 15: `contextA.SaveChanges();` → Envía al proveedor los cambios rastreados y aplica las reglas de transacción/concurrencia configuradas.

Línea 16: `var despuesA = Convert.ToHexString(ordenA.RowVersion);` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 18: `ordenB.Estado = "EnProceso";` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 19: `var conflicto = false;` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 20: `try` → Abre el bloque en el que se espera que pueda producirse la excepción que se quiere estudiar.

Línea 21: `{` → Delimita el bloque sintáctico correspondiente.

Línea 22: `contextB.SaveChanges();` → Envía al proveedor los cambios rastreados y aplica las reglas de transacción/concurrencia configuradas.

Línea 23: `}` → Delimita el bloque sintáctico correspondiente.

Línea 24: `catch (DbUpdateConcurrencyException)` → Captura la excepción específica para aplicar la política de resolución prevista.

Línea 25: `{` → Delimita el bloque sintáctico correspondiente.

Línea 26: `conflicto = true;` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 27: `}` → Delimita el bloque sintáctico correspondiente.

Línea 29: `return new RowVersionM5Dto(` → Devuelve la evidencia recopilada por el escenario para que el caso de uso o el test pueda comprobarla.

Línea 30: `inicialA, inicialB, despuesA, conflicto,` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 31: `SqlCommandCounterInterceptor.Instance.SnapshotCommands();` → Recupera los comandos SQL realmente interceptados durante la operación.

Línea 32: `}` → Delimita el bloque sintáctico correspondiente.

Paso 5: Implementar y estudiar TokensConcurrenciaM5Repositorio.cs

```
public IndiceRowVersionM5Dto ComprobarIndiceRowVersion()
{
    using var scope = _scopeFactory.CreateScope();
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
    var connection = context.Database.GetDbConnection();
    if (connection.State != ConnectionState.Open)
        connection.Open();

    using var command = connection.CreateCommand();
    command.CommandText = """
        SELECT COUNT(*)
        FROM sys.indexes AS i
        INNER JOIN sys.index_columns AS ic
            ON i.object_id = ic.object_id AND i.index_id = ic.index_id
        INNER JOIN sys.columns AS c
            ON ic.object_id = c.object_id AND ic.column_id = c.column_id
        WHERE i.object_id = OBJECT_ID(N'dbo.OrdenesFabricacion')
            AND c.name = N'RowVersion';
        """;

    return new IndiceRowVersionM5Dto(Convert.ToInt32(command.ExecuteScalar()) > 0);
}
```

Explicación línea a línea — TokensConcurrenciaM5Repositorio.cs

Línea 1: `public IndiceRowVersionM5Dto ComprobarIndiceRowVersion()` → Declara la operación pública que ejecutará o verificará este comportamiento.

Línea 2: `{` → Delimita el bloque sintáctico correspondiente.

Línea 3: `using var scope = _scopeFactory.CreateScope();` → Crea un ámbito de DI independiente; los servicios Scoped, incluido `AceriaDbContext`, no se comparten con otros actores.

Línea 4: `var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();` → Resuelve el `DbContext` del ámbito y garantiza que la configuración de Infrastructure sea la misma que en la aplicación.

Línea 5: `var connection = context.Database.GetDbConnection();` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 6: `if (connection.State != ConnectionState.Open)` → Evalúa una condición necesaria y detiene o desvía el flujo cuando no se cumple.

Línea 7: `connection.Open();` → Ejecuta esta instrucción como parte del flujo del ejemplo.

Línea 9: `using var command = connection.CreateCommand();` → Importa el espacio de nombres necesario para los tipos y extensiones usados por el archivo, o declara un recurso con liberación automática cuando se usa `using var`.

Línea 10: `command.CommandText = """` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 11: `SELECT COUNT(*)` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 12: `FROM sys.indexes AS i` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 13: `INNER JOIN sys.index_columns AS ic` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 14: `ON i.object_id = ic.object_id AND i.index_id = ic.index_id` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 15: `INNER JOIN sys.columns AS c` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 16: `ON ic.object_id = c.object_id AND ic.column_id = c.column_id` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 17: `WHERE i.object_id = OBJECT_ID(N'dbo.OrdenesFabricacion')` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 18: `AND c.name = N'RowVersion';` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 19: `"";` → Ejecuta esta instrucción como parte del flujo del ejemplo.

Línea 21: `return new IndiceRowVersionM5Dto(Convert.ToInt32(command.ExecuteScalar()) > 0);` → Devuelve la evidencia recopilada por el escenario para que el caso de uso o el test pueda comprobarla.

Línea 22: `}` → Delimita el bloque sintáctico correspondiente.

Paso 6: Compilar y ejecutar

```
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
dotnet ef migrations list --project src/AceriaData.Infrastructure --startup-project src/AceriaData.Console --configuration Release
dotnet ef database update --project src/AceriaData.Infrastructure --startup-project src/AceriaData.Console --configuration Release
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release --no-build
```

La ejecución debe finalizar con `5.2 OK`. Si una comprobación interna falla, la aplicación lanza una excepción y el punto no se considera correctamente reproducido.

Diagnóstico técnico

La práctica se considera correcta cuando se observan estas condiciones:

- RowVersion se configura explícitamente con `IsRowVersion()`.
- EstadoDetalle se configura como `IsConcurrencyToken()`.
- La migración `M5_5_2_ConcurrencyTokens` existe y se aplica.
- Dos contextos producen `DbUpdateConcurrencyException` cuando cambia el token.
- Se comprueba directamente que RowVersion no obtiene un índice automático.

Reto resuelto y ampliación

Reto: Inspecciona el UPDATE capturado y localiza el token original dentro del predicado de concurrencia.

Para resolverlo, conserva la misma regla del módulo: primero define qué resultado funcional debe mantenerse; después captura SQL, número de comandos, tracking, historial de migraciones o estado transaccional según corresponda. No uses una medición temporal aislada como sustituto de la evidencia técnica.

Errores comunes

- Confiar en el nombre RowVersion sin configurarlo.
- Suponer que rowversion crea automáticamente un índice.
- Probar semántica de rowversion de SQL Server con SQLite.
- Modificar a mano el snapshot o la migración para forzar el resultado.

Cierre acumulativo

1. Ejecuta `dotnet build` en la solución del punto.
2. Ejecuta la aplicación y comprueba el marcador `5.2 OK`.
3. Conserva las migraciones existentes; solo 5.2 añade el cambio de modelo oficial de M5.

4. Comprueba que Application continúa sin depender de Entity Framework Core ni de Infrastructure.
5. Antes de avanzar, relaciona el resultado con el SQL, la transacción, el historial o la prueba que lo demuestra.

Resultado esperado

Al finalizar el punto 5.2, AceriaData debe compilar, ejecutarse sobre SQL Server LocalDB y demostrar de forma observable el comportamiento descrito. El marcador final es:

5.2 OK

Conexión con el siguiente punto

El siguiente estado es 5.3 y parte directamente de este proyecto completo.
